



AAI presents IUT 12th ICT Fest
powered by Therap (BD) Ltd.
organized by IUT Computer Society

Bdapps presents
Agentic AI Hackathon
powered by Codex

Preliminary Round Problem Statement

CoWork: Multi-Tenant Coworking Space Booking API

Repository Link

https://github.com/AlchemistReturns/ICT_Fest_Hackathon_Preliminary

Duration

4 hours, 6:00 PM to 10:00 PM

Document version: Preliminary Round
July 9, 2026

1 Overview

এটি একটি বাগ ফিক্সিং (bug fix) প্রতিযোগিতা। প্রতিযোগীদের একটি ত্রুটিপূর্ণ কোডবেস (broken codebase) দেওয়া হবে এবং তাদের বাগগুলো খুঁজে বের করতে হবে, বুঝতে হবে কেন সেগুলো কাজ করছে না এবং তা সমাধান করতে হবে। পুরো প্রজেক্ট জুড়ে বিভিন্ন ধরনের বাগ লুকানো রয়েছে, যা সহজ এক লাইনের সমাধান থেকে শুরু করে সূক্ষ্ম কনকারেন্সি (concurrency) ও লজিক সংক্রান্ত সমস্যা পর্যন্ত হতে পারে। প্রতিযোগীদের নতুন কোনো ফিচার যোগ করার বা কোড রিফ্যাক্টর (refactor) করার প্রয়োজন নেই। বাগগুলো খুঁজে বের করুন এবং সেগুলো ফিক্স করুন। এটাই সম্পূর্ণ কাজ।

গ্রেডিং প্রক্রিয়াটি সম্পূর্ণ স্বয়ংক্রিয় এবং ব্ল্যাক-বক্স (black-box)। একজন গ্রেডার আপনার করা রিপোজিটরি বিন্দু করবে এবং শুধুমাত্র API-এর মাধ্যমে এটির সাথে যোগাযোগ করবে। এটি এই নথিতে বর্ণিত বিজনেস রুলস (Business Rules) এবং API চুক্তির (API Contract) (সেকশন ৩ এবং ৪) বিপরীতে আচরণ পরীক্ষা করবে। আপনার করা ফিক্সগুলো অবশ্যই এই চুক্তিটি হুবহু বজায় রাখবে - পাথ (paths), স্ট্যাটাস কোড (status codes), এরর কোড (error codes) এবং JSON ফিল্ডের নামগুলো কোনোভাবেই পরিবর্তন করা যাবে না।

2 The Project

কোওয়ার্ক (CoWork) হলো একটি REST API যা একটি কোওয়ার্কিং স্পেসের ভেতরের বুকিংযোগ্য রুমগুলো পরিচালনা করার জন্য তৈরি, যা একাধিক টেন্যান্ট অর্গানাইজেশন (tenant organizations) সমর্থন করে। প্রতিটি অর্গানাইজেশনের নিজস্ব রুম, স্টাফ (এডমিন) এবং মেম্বার রয়েছে। মেম্বাররা নির্দিষ্ট সময়ের জন্য রুম বুক করতে পারে; এডমিনরা রুম পরিচালনা করে এবং ব্যবহারের রিপোর্ট (usage reports) তৈরি করে।

Stack: Python 3.11, FastAPI, SQLAlchemy, SQLite (single file, no external database service).

Authentication: JWT access and refresh tokens (HS256) দ্বারা পরিচালিত হয়।

Out of scope: এখানে কোনো বাস্তব পেমেন্ট গেটওয়ে নেই (রিফান্ড হিসেব এবং লগ করা হয় কিন্তু প্রসেস করা হয় না), এবং কোনো বাস্তব ইমেইল পাঠানোর ব্যবস্থা নেই (একটি "send confirmation email" ধাপ কেবল একটি লাইন লগ করে)।

File Structure

```
app/
|-- main.py           # FastAPI app entrypoint
|-- config.py         # Environment/config loading
|-- database.py       # Database engine and session setup
|-- models.py         # SQLAlchemy database models
|-- schemas.py        # Pydantic request/response schemas
|-- serializers.py    # Model -> response object conversion
|-- auth.py           # JWT creation, password hashing, auth dependency
|-- cache.py          # In-memory caching for reports/availability
|-- errors.py         # Application error types and handler
|-- timeutils.py      # Datetime parsing/normalization helpers
|-- routers/          # API route handlers (auth, rooms, bookings, admin, health)
'-- services/         # Business logic (refunds, stats, rate limiting,
                       # reference codes, export, notifications)
```

3 Data Model

- **Organization:** id, name (unique)
- **User:** id, org_id, username (unique within org), hashed_password, role (admin | member), created_at
- **Room:** id, org_id, name, capacity, hourly_rate_cents
- **Booking:** id, room_id, user_id, start_time, end_time, status (confirmed | cancelled), reference_code, price_cents, created_at
- **RefundLog:** id, booking_id, amount_cents, status (processed | failed), processed_at

4 Business Rules

এপিআই (API) যে নিয়মগুলো মেনে চলবে তা নিচে দেওয়া হলো। কিছু বাগ এই নিয়মগুলোর বাইরে চলে গেছে - কোনো আচরণ সঠিক কিনা তা সিদ্ধান্ত নেওয়ার জন্য এই নিয়মগুলোকে আপনার সত্যের উৎস (source of truth) হিসেবে ব্যবহার করুন।

1. Datetimes. সকল API ডেটটাইম অবশ্যই ISO 8601 ফরম্যাটে হতে হবে। ইনপুট ডেটটাইম যদি UTC অফসেটসহ আসে, তবে সেভ করার আগে বা তুলনা করার আগে অবশ্যই তা UTC-তে রূপান্তর করতে হবে; সাধারণ বা অফসেটহীন (naive) ইনপুটকে UTC হিসেবে গণ্য করা হবে। রেসপন্সের সকল ডেটটাইম একটি সুনির্দিষ্ট UTC নির্দেশকসহ UTC ফরম্যাটে হতে হবে।

2. Booking price. $\text{price_cents} = \text{hourly_rate_cents} \times \text{duration_hours}$ । বুকিংয়ের সময়সীমা (Duration) অবশ্যই পূর্ণ ঘণ্টা হতে হবে, সর্বনিম্ন ১ ঘণ্টা এবং সর্বোচ্চ ৮ ঘণ্টা। `end_time` অবশ্যই `start_time` এর পরে হতে হবে। রিকোয়েস্ট করার সময় `start_time` অবশ্যই ভবিষ্যতের সময় হতে হবে - কোনো গ্রেস উইন্ডো (grace window) দেওয়া হবে না।

3. No double-booking. একই রুমের জন্য দুটি কনফার্মড বুকিং ওভারল্যাপ (overlap) করবে যদি এবং কেবল যদি `existing.start < new.end AND new.start < existing.end` হয়। একটি বুকিং শেষ হওয়ার সাথে সাথে আরেকটি বুকিং (Back-to-back bookings) করা যাবে। কোনো দ্বন্দ্ব বা ওভারল্যাপ হলে → 409 ROOM_CONFLICT রিটার্ন করবে। কনকারেন্ট রিকোয়েস্টের (concurrent requests) ক্ষেত্রেও এই নিয়ম কার্যকর হতে হবে।

4. Booking quota. একজন মেম্বার তার অর্গানাইজেশনের সকল রুম মিলিয়ে `(now, now + 24h]` এই সময়সীমার মধ্যে `start_time` আছে এমন সর্বোচ্চ ৩টি কনফার্মড বুকিং রাখতে পারবেন। এর বেশি হলে → 409 QUOTA_EXCEEDED রিটার্ন করবে। কনকারেন্ট রিকোয়েস্টের ক্ষেত্রেও এই নিয়ম কার্যকর হতে হবে।

5. Rate limit. POST /bookings রিকোয়েস্ট প্রতি ইউজারের জন্য প্রতি রোলিং ৬০ সেকেন্ডে সর্বোচ্চ ২০টি রিকোয়েস্টে সীমাবদ্ধ (সব ধরনের রিকোয়েস্টই এর অন্তর্ভুক্ত হবে)। এর বেশি হলে → 429 RATE_LIMITED রিটার্ন করবে। কনকারেন্ট রিকোয়েস্টের ক্ষেত্রেও এই নিয়ম কার্যকর হতে হবে।

6. Cancellation refund policy. শুধুমাত্র বুকিংয়ের মালিক অথবা একই অর্গানাইজেশনের এডমিন বুকিং বাতিল করতে পারবেন। নোটিশের সময় = `start_time` - বুকিং বাতিলের সময় :

- `notice >= 48 hours` → 100% refund
- `24 hours <= notice < 48 hours` → 50% refund
- `notice < 24 hours` → 0% refund

রিফান্ডের পরিমাণ নিকটবর্তী সেন্টে (nearest cent) রাউন্ড করতে হবে, অর্ধেক সেন্টের (.5 cent) ক্ষেত্রে উপরের দিকে (rounding up) রাউন্ড হবে। ইতিমধ্যে বাতিল করা কোনো বুকিং পুনরায় বাতিল করতে গেলে → 409 ALREADY_CANCELLED রিটার্ন করবে। একটি বাতিল হওয়া বুকিংয়ের জন্য ঠিক একটি RefundLog এন্ট্রি থাকবে, এবং বাতিল করার রেসপন্সে যে রিফান্ড অ্যামাউন্ট পাঠানো হবে তা অবশ্যই RefundLog -এ সংরক্ষিত অ্যামাউন্টের সমান হতে হবে। একই বুকিংয়ের জন্য একাধিক কনকারেন্ট বুকিং বাতিলের রিকোয়েস্টের ক্ষেত্রেও এটি নিশ্চিত করতে হবে।

7. Reference codes. প্রতিটি বুকিংয়ের `reference_code` অবশ্যই ইউনিক (অনন্য) হতে হবে, এমনকি কনকারেন্ট বুকিং তৈরির সময়ও।

8. Auth. টোকেনগুলো হলো JWT (HS256) যার ক্লেইমগুলো হলো `sub` (user id, string), `org` (org id), `role`, `jti` (টোকেনের জন্য ইউনিক), `iat`, `exp`, `type` (access | refresh)। এক্সেস টোকেন ঠিক ৯০০ সেকেন্ড পর এক্সপায়ার হবে। রিফ্রেশ টোকেন ৭ দিন পর এক্সপায়ার হবে। লগআউট করার সাথে সাথে ব্যবহৃত এক্সেস টোকেনটি বাতিল (invalidate) হয়ে যাবে (পরবর্তীতে এটি ব্যবহার করলে → 401 রিটার্ন করবে)। রিফ্রেশ টোকেন শুধুমাত্র একবারই ব্যবহারযোগ্য (single-use): রিফ্রেশ করলে একটি নতুন এক্সেস এবং রিফ্রেশ টোকেন রিটার্ন করবে এবং পূর্বের রিফ্রেশ টোকেনটি বাতিল করে দিবে (পুনরায় ব্যবহার করলে → 401 রিটার্ন করবে)।

9. Multi-tenancy. একজন ইউজার (এডমিনসহ) প্রতিটি কোড পাথে শুধুমাত্র তার নিজস্ব অর্গানাইজেশনের ডাটা দেখতে বা কাজ করতে পারবেন। অন্য অর্গানাইজেশনের রিসোর্স আইডিগুলোর ক্ষেত্রে এমন আচরণ করবে যেন সেগুলোর কোনো অস্তিত্ব নেই (→ 404)।

10. Booking visibility. মেম্বাররা শুধুমাত্র নিজেদের বুকিং দেখতে এবং বাতিল করতে পারবেন (অন্য মেম্বারের বুকিং আইডি হলে → 404 BOOKING_NOT_FOUND রিটার্ন করবে)। এডমিনরা তাদের অর্গানাইজেশনের যেকোনো বুকিং দেখতে এবং বাতিল করতে পারবেন।

11. Pagination & ordering. GET /bookings রিকোয়েস্ট `page` (ডিফল্ট ১) এবং `limit` (ডিফল্ট ১০, সর্বোচ্চ ১০০) প্যারামিটার গ্রহণ করে। আইটেমগুলো হবে রিকোয়েস্টকারীর নিজস্ব বুকিং যা `start_time` অনুযায়ী উর্ধ্বক্রমে (ascending) সাজানো থাকবে (একই সময় হলে ascending id অনুযায়ী সাজানো হবে)। ক্রমানুসারে আসা পেজগুলোতে কোনো আইটেম বাদ যাবে না বা পুনরাবৃত্তি হবে না। রেসপন্সে `total` বুকিংয়ের সংখ্যা অন্তর্ভুক্ত থাকতে হবে।

12. Usage report. GET /admin/usage-report?from=...&to=... রিকোয়েস্টটি রিকোয়েস্টকারীর অর্গানাইজেশনের প্রতি রুমের জন্য (বুকিং না হওয়া রুমসহ), [from, to] (UTC, inclusive) সময়ের মধ্যে শুরু হওয়া কনফার্মড বুকিংয়ের সংখ্যা এবং মোট price_cents রিটার্ন করবে। এটি তাৎক্ষণিকভাবে বর্তমান অবস্থা প্রতিফলিত করতে হবে।

13. Availability. GET /rooms/{id}/availability?date=... রিকোয়েস্টটি উক্ত UTC তারিখে রুমটির সমস্ত কনফার্মড বুকিং ব্যস্ত সময়সীমা (busy intervals) হিসেবে উর্ধ্বক্রমে সাজিয়ে রিটার্ন করবে, যা তাৎক্ষণিকভাবে বর্তমান অবস্থা প্রতিফলিত করবে।

14. Room stats. GET /rooms/{id}/stats রিকোয়েস্টটি রুমের বর্তমান কনফার্মড বুকিংয়ের সংখ্যা এবং তাদের মোট price_cents রিটার্ন করবে, যা সর্বদা বুকিং ডাটার সাথে সামঞ্জস্যপূর্ণ হবে (এমনকি কনকারেন্ট অ্যাক্টিভিটি বৃদ্ধির পরেও)।

15. Registration. POST /auth/register রিকোয়েস্টে অজানা/নতুন org_name পাঠানো হলে অর্গানাইজেশনটি তৈরি হবে এবং ইউজার এডমিন (admin) হিসেবে নিবন্ধিত হবে; পরিচিত/বিদ্যমান org_name হলে ইউজার মেম্বর (member) হিসেবে যোগ দেবে। একই অর্গানাইজেশনে ডুপ্লিকেট ইউজারনেম হলে → 409 USERNAME_TAKEN রিটার্ন করবে।

16. Liveness. সার্ভিসটিকে সব সময় সকল এন্ডপয়েন্টে রেসপন্স করতে হবে; কোনো কনকারেন্ট বৈধ রিকোয়েস্টের কন্সিশনেশন সার্ভিসটিকে হ্যাং বা বন্ধ করে দিতে পারবে না।

5 API Contract

Endpoints

Method	Path	Auth	Description
POST	/auth/register	No	Register org admin or join org as member
POST	/auth/login	No	Returns access + refresh token
POST	/auth/refresh	No (token in body)	Rotates tokens
POST	/auth/logout	Yes	Invalidates presented access token
GET	/rooms	Yes	List rooms in caller's org
POST	/rooms	Yes (admin)	Create a room
GET	/rooms/{id}/availability	Yes	Busy intervals for a date
GET	/rooms/{id}/stats	Yes	Live confirmed-booking count & revenue
POST	/bookings	Yes	Create a booking
GET	/bookings	Yes	Caller's bookings, paginated
GET	/bookings/{id}	Yes	Single booking incl. refunds
POST	/bookings/{id}/cancel	Yes	Cancel + refund calculation
GET	/admin/usage-report	Yes (admin)	Per-room usage/revenue for range
GET	/admin/export	Yes (admin)	Bookings CSV; room_id, include_all
GET	/health	No	{"status": "ok"}

অনুমোদিত এন্ডপয়েন্টগুলোর (authenticated endpoints) জন্য Authorization হেডারে টোকেনটি পাঠাতে হবে:

```
Authorization: Bearer <your_token>
```

Request / Response Schemas

- **POST /auth/register** body {org_name, username, password} → {user_id, org_id, username, role}
- **POST /auth/login** body {org_name, username, password} → {access_token, refresh_token, token_type: "bearer"}; bad credentials → 401 INVALID_CREDENTIALS
- **POST /auth/refresh** body {refresh_token} → same shape as login
- **Room:** {id, org_id, name, capacity, hourly_rate_cents}; **POST /rooms** body {name, capacity, hourly_rate_cents}
- **Availability:** {room_id, date, busy: [{start_time, end_time}, ...]}
- **Stats:** {room_id, total_confirmed_bookings, total_revenue_cents}
- **POST /bookings** body {room_id, start_time, end_time} → Booking: {id, reference_code, room_id, user_id, start_time, end_time, status, price_cents, created_at}
- **GET /bookings** → {items: [Booking, ...], page, limit, total}
- **GET /bookings/{id}** → Booking plus refunds: [{amount_cents, status, processed_at}, ...]
- **POST /bookings/{id}/cancel** → {id, status: "cancelled", refund_percent, refund_amount_cents}
- **Usage report** → {from, to, rooms: [{room_id, room_name, confirmed_bookings, revenue_cents}, ...]}
- **Export CSV header (exact):**
id,reference_code,room_id,user_id,start_time,end_time,status,price_cents

Errors

অ্যাপ্লিকেশন এররগুলো JSON {"detail": <string>, "code": <CODE>} রিটার্ন করবে যেখানে কোডগুলো হবে: USERNAME_TAKEN (409), INVALID_CREDENTIALS (401), ROOM_CONFLICT (409), QUOTA_EXCEEDED (409), RATE_LIMITED (429), ALREADY_CANCELLED (409), BOOKING_NOT_FOUND (404), ROOM_NOT_FOUND (404), FORBIDDEN (403), INVALID_BOOKING_WINDOW (400 — অতীতের সময়, অপূর্ণ/সীমার বাইরে বুকিংয়ের সময়সীমা, অথবা end_time <= start_time)। টোকেন অনুপস্থিত/ত্রুটিপূর্ণ/মেয়াদোত্তীর্ণ/ব্ল্যাকলিস্টেড হলে → 401 রিটার্ন করবে। ফ্রেমওয়ার্ক ভ্যালিডেশন এররগুলো (422) FastAPI-এর ডিফল্ট ফরম্যাট ব্যবহার করতে পারে।

ফিক্সগুলো অবশ্যই এই চুক্তিটি ছবছ বজায় রাখবে; এর বিপরীতে ব্ল্যাক-বক্স প্রেডিং করা হবে।

6 Running the Project

With Docker (recommended)

```
docker compose up --build
```

Without Docker (Python 3.11)

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
uvicorn app.main:app --reload
```

অ্যাপটি <http://localhost:8000> ঠিকানায় চলবে। ইন্টারঅ্যাক্টিভ API ডকুমেন্টেশন পাবেন <http://localhost:8000/docs> লিংকে।

7 How to Test

Swagger UI (</docs> লিংকে), curl, অথবা Postman-এর মতো ক্লায়েন্ট ব্যবহার করুন।

curl Example

```
# Register
curl -X POST http://localhost:8000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"org_name": "acme", "username": "alice", "password": "pass123"}'

# Login
curl -X POST http://localhost:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"org_name": "acme", "username": "alice", "password": "pass123"}'

# Use the token
curl http://localhost:8000/rooms \
  -H "Authorization: Bearer <TOKEN>"
```

8 The Challenge

কোডবেসে একাধিক বাগ রয়েছে, যা নিচের বিভিন্ন ডিফিকাল্টি লেভেলে বিন্যস্ত। প্রতিটি বাগ এপিআই (API)-এর সাথে যোগাযোগের সময় স্পষ্ট ও দৃশ্যমান ভুল আচরণ প্রদর্শন করে।

বৈধ ফিক্স (valid fix) হিসেবে যা গণ্য হবে:

- ফিক্স করা কোডটি অবশ্যই সেকশন ৩-৪ এ বর্ণিত সঠিক আচরণ প্রদর্শন করবে।
- শুধুমাত্র ত্রুটিপূর্ণ কোড পরিবর্তন করতে হবে - অসম্পর্কিত কোড রিফ্যাক্টর বা রিরাইট (পুনরায় লেখা) করবেন না।
- এপিআই চুক্তি বা এপিআই কন্ট্রাক্ট (পাথ, স্ট্যাটাস কোড, এরর কোড, JSON ফিল্ডের নাম) অবশ্যই হুবহু বজায় থাকতে হবে।

9 Submission

১. প্রিলিমিনারি রাউন্ডের রিপোজিটরিটি আপনার নিজস্ব গিটহাব অ্যাকাউন্টে ফোর্ক (Fork) করুন: https://github.com/AlchemistReturns/ICT_Fest_Hackathon_Preliminary
২. ফোর্কের নেটওয়ার্ক বিচ্ছিন্ন করুন যাতে আপনার কপিটি মূল রিপোজিটরির সাথে সংযুক্ত না থাকে (GitHub → your repo's Settings → scroll to Danger Zone → Leave fork network)। কোড এডিট করার আগেই এটি নিশ্চিত করুন।
৩. আপনার রিপোজিটরিতে বাগগুলো ফিক্স করুন।
৪. আপনি চাইলে প্রতিযোগিতার সময় আপনার রিপোজিটরিটি প্রাইভেট (private) রাখতে পারেন।
৫. প্রতিযোগিতা শেষ হওয়ার ১ ঘণ্টার মধ্যে আপনাকে রিপোজিটরিটি পাবলিক (public) করতে হবে - এই সময়ের পরেও যে রিপোজিটরিগুলো প্রাইভেট থাকবে সেগুলো মূল্যায়ন করা হবে না।
৬. প্রদত্ত গুগল ফর্ম লিংকের মাধ্যমে আপনার রিপোজিটরি URL সাবমিট করুন।

10 Scoring & Tie-Breaking

কঠিনতার ওপর ভিত্তি করে প্রতিটি বাগের জন্য পয়েন্ট প্রদান করা হবে:

Difficulty	Points each
Easy	3
Medium	5
Hard	10

Tie-Breaking

টাই হলে নিম্নলিখিত ক্রমানুসারে নিষ্পত্তি করা হবে:

১. **সমাধান করা বাগের কঠিনতা** (Difficulty of bugs solved) - যে প্রতিযোগী তুলনামূলক কঠিন বাগ ফিক্স করেছেন তিনি এগিয়ে থাকবেন।
২. **bug_report.md (ঐচ্ছিক)** - প্রথম ধাপের পরেও টাই থাকলে, যে প্রতিযোগীরা তাদের রিপোজিটরির রুটে `bug_report.md` সাবমিট করেছেন তাদের ম্যানুয়াল ইভালুয়েশন (হাতে কলমে মূল্যায়ন) করা হবে।

bug_report.md should include, for each bug found:

- Which file(s)/line(s) the bug is on
- What the bug was and why it caused incorrect behavior
- How it was fixed

টাই নিষ্পত্তির জন্য `bug_report.md` এর ম্যানুয়াল ইভালুয়েশনই শেষ প্রক্রিয়া।

Good luck.